

- Multi-Label Legal Classification Project
 - Project Structure
 - Setup
 - Training
 - Standard Models
 - Hierarchical Variants
 - Evaluation Only
 - RAG for Classification (k-NN)
 - Advanced Data Efficiency Experiments (Scaling)
 - 1. Strict Fairness Protocol
 - 2. Training Strategy (Dynamic Epochs)
 - 3. Cost Analysis (FLOPs Estimation)
 - Inference
 - Technical Implementation: Guiding Models to Output Labels 0-9
 - Technical Details: Fine-Tuning & Memory Optimization
 - Notes on Input Strategies
 - Standard Input Window
 - Hierarchical Input
 - Models Details

Multi-Label Legal Classification Project

This project implements multi-label classification on the **ecthr_a** subset of the **coastalcph/lex_glue** dataset using Hugging Face Transformers.

It compares two main approaches:

1. **Standard Sequence Classification:** Truncating documents to fit model context.
2. **Hierarchical Classification:** Splitting documents into segments and aggregating representations ("Divide and Conquer").

Models used:

- **nlpaueb/legal-bert-base-uncased**
- **Qwen/Qwen3-Embedding-0.6B**
- **Qwen/Qwen3-Embedding-4B**
- **Qwen/Qwen3-Embedding-8B**

Project Structure

- **configs/**: Configuration files for training.
- **scripts/**: Shell scripts to launch training and evaluation.
- **models/**: Custom model definitions (e.g., HierarchicalClassifier).
- **train.py**: Main training and evaluation script.
- **inference.py**: Script for inference on new text.
- **requirements.txt**: Python dependencies.
- **.gitignore**: Git ignore rules (excluding models and envs).

Setup

1. Install dependencies:

```
pip install -r requirements.txt
```

2. Login to services (Required for tracking and model upload):

```
# Login to Hugging Face (requires Write token)
huggingface-cli login

# Login to Weights & Biases
wandb login
```

3. Install Flash Attention (Required for Qwen models):

```
pip install flash-attn --no-build-isolation
```

Training

Training scripts are configured to:

1. Use **Weights & Biases** (**wandb**) for experiment tracking.

2. Automatically **upload the trained model** to your Hugging Face Hub account upon completion.

Standard Models

To train and evaluate the standard BERT model (max 512 tokens):

```
./scripts/run_bert.sh
```

To train and evaluate the standard Qwen model (max 2048 tokens):

```
./scripts/run_qwen.sh
```

Hierarchical Variants

The hierarchical approach implements a "Divide and Conquer" strategy specifically designed for long legal documents:

1. **Segmentation:** The document is split into multiple segments (paragraphs).
2. **Encoding (Layer 1):** The base model (Legal-BERT or Qwen) independently encodes each segment into a vector.
 - *Legal-BERT*: Uses the **[CLS]** token.
 - *Qwen*: Uses the last valid token embedding.
3. **Aggregation (Layer 2):** A Transformer Encoder layer processes the sequence of segment vectors to capture document-level context.
4. **Classification:** The aggregated representation is used for final multi-label prediction.

To train Hierarchical Legal-BERT:

```
./scripts/run_hierarchical_bert.sh
```

To train Hierarchical Qwen-Embedding:

```
./scripts/run_hierarchical_qwen.sh
```

Evaluation Only

To evaluate both Baseline (untrained) and Fine-tuned models on the test split:

```
./scripts/eval_all.sh
```

This script sequentially runs evaluation for all 8 configurations (Standard/Hierarchical x BERT/Qwen x Baseline/Fine-tuned).

RAG for Classification (k-NN)

We also support a **Retrieval-Augmented Classification** approach (k-Nearest Neighbors). Instead of training a classifier head, this method:

1. Encodes the entire **Training Set** into a vector store (Knowledge Base).
2. Encodes each **Test Document** (Query).
3. Retrieves the top- k most similar training documents.
4. Predicts labels using a **weighted vote** of the neighbors' labels (weighted by cosine similarity).

This effectively treats the training set as an external memory, allowing for non-parametric classification which can be particularly effective in low-resource or zero-shot scenarios.

Advanced Data Efficiency Experiments (Scaling)

We conduct a rigorous comparison between **Training-Free RAG** and **Efficient Fine-Tuning** across multiple data scales (100, 500, 1000, 2000, 4500, 9000 samples).

To run the Full Data Scaling Experiment on 4 GPUs:

```
# GPU 0: BERT Large Scale (2000-9000 samples)
./scripts/run_gpu0.sh

# GPU 1: BERT Small Scale (100-1000 samples)
./scripts/run_gpu1.sh

# GPU 2: RAG 8B Scaling (All sizes)
./scripts/run_gpu2.sh

# GPU 3: RAG 0.6B + 4B Scaling (All sizes)
./scripts/run_gpu3.sh
```

1. Strict Fairness Protocol

To ensure a fair comparison between methods, we enforce strict data consistency:

- **Seed Control:** Both RAG and BERT experiments use **seed=42**.
- **Subset Consistency:** The subset of 100 samples used for RAG is **identically** the same subset used for BERT fine-tuning (achieved by shuffling the full dataset with the same seed before slicing).
- **Long Context:** Both methods are evaluated with **max_seq_length=8192**.
 - **RAG:** Uses native long-context capabilities of Qwen3-Embedding.
 - **BERT:** Uses Hierarchical Architecture (64 segments x 128 tokens) with LoRA.

2. Training Strategy (Dynamic Epochs)

For fine-tuning, we adapt the number of training epochs based on dataset size to balance convergence and overfitting:

- **Small Data (≤ 500):** 20 Epochs (to maximize learning from limited data).
- **Medium Data (1000):** 10 Epochs.
- **Large Data (≥ 2000):** 5 Epochs.

3. Cost Analysis (FLOPs Estimation)

We estimate the computational cost to demonstrate the efficiency gap.

Assumptions for Calculation:

- **Model:** Hierarchical Legal-BERT (~116M params total).
- **Sequence Length:** 8192 tokens.
- **Training:** 2000 samples $\times$ 5 epochs.

- **Inference (RAG):** 1000 test samples.

Method	FLOPs (Est.)	Relative Cost	Memory (Optimizer States)
RAG (Inference Only)	1.9e15 (1.9 PetaFLOPs)	1x	0 MB
LoRA Fine-tuning	3.8e16 (38 PetaFLOPs)	~20x	~50 MB
Full Fine-tuning	5.7e16 (57 PetaFLOPs)	~30x	~928 MB

Key Insight: Even with LoRA, fine-tuning requires backpropagation through the full 8k context, resulting in FLOPs typically **20-30 times higher** than the RAG approach. Furthermore, RAG requires **zero training memory**, making it deploying on commodity hardware much easier.

- **Fine-Tuning FLOPs Formula:** $C \approx 6 \times N_{params} \times S_{samples} \times E_{epochs} \times L_{seq}$
- **RAG Inference FLOPs Formula:** $C \approx 2 \times N_{params} \times S_{test} \times L_{seq}$

Inference

To use a trained model for prediction on a text string:

```
python inference.py ./output/bert_ecthr_a "Your legal text here..."
```

Technical Implementation: Guiding Models to Output Labels 0-9

The models are guided to output labels 0-9 using a **Classification Head** and **BCE Loss**:

1. **Architecture:** A linear layer (head) is added on top of the model's final embedding. It maps the hidden state to **10 output logits**, representing classes 0-9.
2. **Training Target:** Raw labels (e.g., `[2, 5]`) are converted to a **Multi-hot Vector** (e.g., `[0, 0, 1, 0, 0, 1, 0, 0, 0, 0]`).

- 3. **Loss Function:** We use **BCEWithLogitsLoss**. It treats each of the 10 outputs as an independent binary classification problem (Is label 0 present? Is label 1 present? ...).
- 4. **Inference:** The model outputs logits. We apply **Sigmoid** to get probabilities and output any label with probability > **0.5**.

Technical Details: Fine-Tuning & Memory Optimization

We observed distinct memory behaviors between Standard Classification (SFT) and Embedding Fine-Tuning. Here is a comparison of the technical configurations required to run these experiments on standard hardware (e.g., A10 24GB vs. A100 40GB):

Feature	Standard SFT (e.g., Qwen-0.6B)	Embedding Fine-Tuning	Impact on Memory
Loss Function	Cross Entropy (BCE)	InfoNCE (Contrastive)	Embedding requires storing activations for pairs (Anchor + Positive), effectively doubling sequence memory usage.
Input Structure	Single Sequence	Pairs (Anchor, Positive)	Contrastive learning necessitates processing two distinct sequences per sample.
Max Sequence Length	2048 - 8192	2048 - 8192	Memory usage grows quadratically ($O(L^2)$) with length. 8192 is significantly more demanding than 2048.
Gradient Checkpointing	Enabled	Enabled (Critical)	Essential for long sequences. Trades compute for memory (re-computes activations during backward pass). Required for Qwen @ 2048+ length on A100.
Batch Size Strategy	Can be 1 (with Accumulation)	Must be > 1	Contrastive loss needs in-batch negatives. If Batch Size=1, the

Feature	Standard SFT (e.g., Qwen-0.6B)	Embedding Fine-Tuning	Impact on Memory
			model cannot learn from negatives.

Key Takeaway for Embedding Fine-Tuning: To fine-tune embeddings with long contexts (2048+) on GPUs:

1. **Gradient Checkpointing** is mandatory.
2. **Flash Attention 2** is mandatory.
3. **Batch Size** is constrained by memory but must be at least 2 (per GPU) for contrastive learning to be effective.

Notes on Input Strategies

Standard Input Window

- **BERT:** Inputs are truncated to 512 tokens.
- **Qwen:** Inputs are truncated to 2048 tokens (configurable up to ~32k, but limited by VRAM).

Hierarchical Input

- **Hierarchical BERT:** Documents are split into up to 32 segments of 128 tokens each.
- **Hierarchical Qwen:** Documents are split into up to 16 segments of 512 tokens each.
- This architecture allows the models to process documents far longer than their native context windows.

Models Details

- **Legal-BERT:** A specialized BERT model pre-trained on legal text, used here as the segment encoder in the hierarchical setup.

- **Qwen-Embedding:** A decoder-based embedding model. In the hierarchical setup, it functions as a powerful feature extractor for each segment, proving its versatility as a "first layer" encoder despite its decoder architecture.